

Implémentation d'un CNN en C et CUDA pour la reconnaissance de chiffres manuscrits

Travail de maturité

Nathan Gasser

9 juillet 2026

Résumé

Les réseaux de neurones à convolution (CNN) sont très utilisés de nos jours pour la reconnaissance d'images. Nous allons en implémenter un en utilisant les langages C et CUDA. Son but sera de lire un chiffre manuscrit et sera entraîné sur le dataset MNIST, en utilisant une carte graphique NVIDIA.

Table des matières

1	Introduction	3
2	Modèle de reconnaissance d'images	3
3	MLP	3
3.1	Perceptron	3
3.2	Fonctions d'activation	4
3.2.1	La fonction <i>ReLU</i>	4
3.2.2	La fonction <i>tanh</i>	5
3.3	Plusieurs neurones	5
4	Backpropagation	6
5	CNN	6
5.1	Convolution	6
5.2	Pooling	6
5.3	CNN complet	6
6	Implémentation en C et CUDA	6
6.1	Moteur d'inférence en C	6
6.2	Moteur d'entraînement en CUDA	6
6.3	Interface utilisateur	6

1 Introduction

Pour la reconnaissance de chiffres manuscrits, il y a plusieurs méthodes possibles. Une des méthodes les plus simples est le perceptron à plusieurs couches (MLP), mais les CNN sont plus efficaces pour les tâches de reconnaissance d'images. Un CNN est un type de réseau de neurones qui a pour particularité d'utiliser des opérations de convolution pour extraire des caractéristiques de l'image. Ce TM aura pour but d'implémenter ce type de réseau de neurones en CUDA pour la partie entraînement et par la suite en C pour la partie inférence.

2 Modèle de reconnaissance d'images

Pour faire un modèle qui est capable de lire un chiffre manuscrit, il faut d'abord comprendre sous quelle forme donner l'image du chiffre manuscrit et également sous quelle forme se présentera la sortie du modèle.

Chaque chiffre manuscrit est une image de 28×28 pixels avec à chaque fois 256 différentes intensités de noir. C'est à dire que si la valeur du pixel est de 0, il correspond à un pixel blanc et si la valeur du pixel est de 255, il correspond à un pixel noir. Les valeurs entre deux correspondent à différentes intensités de gris. Notez que la valeur maximale d'un pixel correspond à 255 et non 256, tout simplement parce qu'il y a 256 valeurs différentes et que la valeur 0 est incluse, ce qui nous laisse 255 valeurs positives. Nous allons ensuite les diviser tous par 255, ce qui nous permet d'avoir des valeurs entre 0 et 1.0 que nous allons pouvoir donner au modèle. Nous avons donc $28 \times 28 = 784$ pixels dans une image. Notre modèle aura donc 784 entrées avec des valeurs comprises entre 0 et 1.0 à chaque fois.

Nous voulons obtenir en sortie 10 probabilités pour chacun des 10 chiffres possibles (de 0 à 9). Il y aura donc 10 neurones en sortie, chacun des différents chiffres sera assigné à un neurone et les valeurs de sortie des neurones se situeront entre -1.0 et 1.0 . Une valeur de -1.0 correspond à une certitude qu'il ne s'agit pas du chiffre associé à l'image et 1.0 correspond à une certitude qu'il s'agit du bon chiffre.

3 MLP

L'architecture la plus simple qu'on peut utiliser est le perceptron à plusieurs couches (MLP), il est composé de plusieurs couches composées elles même de plusieurs perceptrons. Bien qu'elle soit ancienne, elle est toujours utilisée elle même à l'intérieur de beaucoup d'autres architectures comme le CNN.

3.1 Perceptron

Le perceptron est la brique de base de l'intelligence artificielle. Il accepte un nombre fixe d'entrées (inputs) (N) et nous donne un seul nombre en sortie (output). Le perceptron peut en lui même déjà être utilisé pour créer des systèmes de classification simples.

Chaque entrée s'appelle x_n (où n est un chiffre entre 0 et N non inclu). Pour chaque entrée, il y a un poids (weight) associé (w_n). En plus des entrées et des poids, il y a un biais (bias) (b) associé à chaque neurone. Les poids et les biais sont initialisés de manière aléatoire. Pour calculer la valeur de sortie, il faut également choisir une fonction d'activation (f). Il peut s'agir théoriquement de n'importe quelle fonction mathématique qui n'accepte qu'une seule entrée, mais dans le cadre de ce TM, nous allons utiliser deux fonctions principalement : ReLU et tanh

Pour calculer la valeur de sortie de notre perceptron, il faudra faire une somme pondérée de nos entrées à laquelle nous allons additionner le biais avant de faire passer cette valeur dans la fonction d'activation :

$$y = f\left(\sum_{n=1}^N w_n x_n + b\right)$$

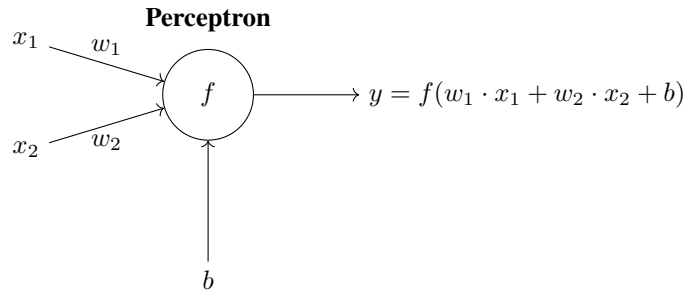


FIGURE 1 – Schéma d'un perceptron avec deux entrées (x_1 et x_2), deux poids (w_1 et w_2), un biais (b) et une fonction d'activation (f) qui produit la sortie (y).

3.2 Fonctions d'activation

La fonction d'activation permet d'ajouter de la non linéarité au modèle. Cela permet au modèle d'apprendre des motifs bien plus complexes. Sans fonction d'activation, le modèle ne pourrait apprendre que des motifs linéaires, ce qui est beaucoup trop limité pour des tâches de reconnaissance d'images.

3.2.1 La fonction *ReLU*

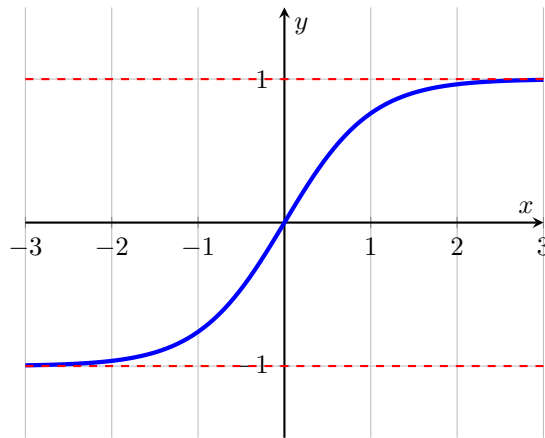
La fonction *ReLU* est la suivante :

$$ReLU(x) = \begin{cases} 0 & \text{si } x < 0 \\ x & \text{si } x \geq 0 \end{cases}$$

3.2.2 La fonction $\tanh$

La fonction $\tanh$ (tangente hyperbolique) est utilisée pour confiner les valeurs de sorties entre la valeur -1.0 et 1.0 . Dans le contexte de la reconnaissance d'images, elle permet de représenter le niveau de certitude de notre modèle par rapport à différentes prédictions. Les fonctions softmax ou sigmoid sont également souvent utilisées mais par simplicité, j'ai décidé d'utiliser la fonction $\tanh$.

Graphe de $\tanh(x)$



3.3 Plusieurs neurones

Pour former un MLP complet, il faut agencer plusieurs couches de plusieurs perceptrons ensemble. A part pour la couche initiale du modèle, la valeur de l'entrée numéro i dans le neurone numéro n dans la couche numéro c correspondra à la valeur de sortie du neurone $n - 1$ sur la couche $c - 1$. Si nous sommes sur la couche initiale du réseau, les entrées des différents neurones correspondent aux entrées du modèle (dans notre cas, les différents pixels d'une image). Pour les couches initiales et finales, il s'agit des couches d'entrées et de sorties respectivement.

Nous pouvons schématiser un MLP ainsi :

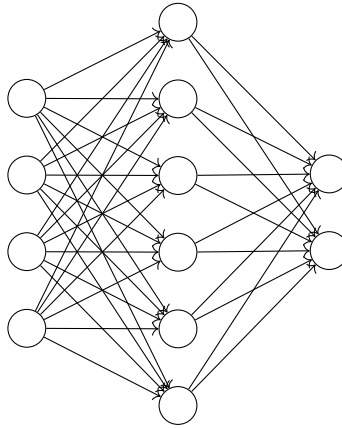


FIGURE 2 – Schéma d'un MLP avec 4 entrées et 2 sorties

4 Backpropagation

Pour que notre modèle puisse correctement prédire les sorties correctes, il va falloir l'entraîner. Sans entraînement, les prédictions seront tout simplement aléatoires. Pour entraîner notre modèle, nous allons utiliser la backpropagation qui est considéré comme l'algorithme le plus important dans le domaine du deep learning.

4.1 Erreur

En utilisant la backpropagation, notre but sera de minimiser la perte (loss) (L) du modèle. Si avec une certaine entrée, on obtient pour la sortie numéro n une valeur de y_n , alors que nous voulions obtenir une valeur de l_n , nous pouvons calculer la perte en soustrayant la valeur que nous attendions à la valeur que nous avons obtenu et élever le tout au carré.

$$L = (y_n - l_n)^2$$

5 CNN

5.1 Convolution

5.2 Pooling

5.3 CNN complet

6 Implémentation en C et CUDA

6.1 Moteur d'inférence en C

6.2 Moteur d'entraînement en CUDA

6.3 Interface utilisateur